CONDUIT CONTROL CENTER

Product & Work Breakdown Structure

v0.1 MVP Implementation Plan

May 31, 2026

Based on SRS v1.1

Total Estimate: 41–52 days

Raspberry Pi 4 · Ubuntu 22.04 ARM64 · MIT Licence

1. Introduction

This document defines the Product Breakdown Structure (PBS) and Work Breakdown Structure (WBS) for the Conduit Control Center v0.1 MVP. It translates the requirements in SRS v1.1 Section 3 into deliverable components and actionable tasks with effort estimates and dependencies.

1.1 How to Read This Document

- The PBS (Section 2) defines what the product is made of — components and deliverables.
- The WBS (Section 3) defines the work needed to build those components — tasks with estimates.
- Section 4 maps WBS tasks to GitHub Issues.
- Effort estimates are in developer-days (one day = 6–7 focused hours). Ranges reflect uncertainty.
- Dependencies show which tasks must be completed before a task can begin.

1.2 Effort Summary

ID	Epic	Tasks	Min Days	Max Days
E1	Project Setup & Repository	5	3	4
E2	Infrastructure & Deployment	7	8	10
E3	Backend — Authentication	5	4	5
E4	Backend — Conduit Adapter	4	4	5
E5	Backend — Metrics & Logs	4	4	5
E6	Frontend	8	8	10
E7	Security Hardening	4	3	4
E8	Testing	3	4	5
E9	Documentation	3	4	3
TOTAL	9 Epics, 43 Tasks	43	40	50

⚠ Estimates assume a single developer working alone. A two-person team can parallelize E3–E6 and reduce total elapsed time to roughly 4–5 weeks.

2. Product Breakdown Structure (PBS)

The PBS shows what the product is made of. Each node is a concrete deliverable that can be tested and verified independently.

ℹ Conduit Control Center v0.1 is composed of 9 product components. The tree below shows how they relate.

L	Code	Deliverable / Component
0	**CCC v0.1**	**Conduit Control Center — complete MVP release**
1	**P1 — Repository**	**Version-controlled source code, CI pipeline, issue templates**
	P1.1	GitHub repository with branch protection and CI/CD configured
	P1.2	GitHub Actions workflow (lint, test, security audit)
	P1.3	Issue and pull request templates
	P1.4	Development environment documentation
1	**P2 — Infrastructure**	**Server-side deployment artefacts that host the application**
	P2.1	install.sh — automated installation script for Ubuntu 22.04 ARM64
	P2.2	uninstall.sh — clean removal script
	P2.3	conduit-cc.service — systemd service unit
	P2.4	conduit-cc.conf — Nginx virtual host with HTTPS
	P2.5	TLS certificate (Cloudflare Origin Cert or Let's Encrypt)
	P2.6	UFW firewall rules (ports 22, 80, 443 only)
	P2.7	cloudflare-ddns.sh — DDNS update script; cron job entry; ddns.log output file
1	**P3 — Backend API**	**FastAPI application serving authenticated REST endpoints**
	P3.1	Application skeleton (FastAPI app, config loader, .env support)
	P3.2	Authentication module (login, logout, session, lockout)
	P3.3	Settings module (change password, session timeout config)
	P3.4	Conduit adapter module (status, start, stop, restart, pairing)
	P3.5	Metrics module (system health, traffic counters)
	P3.6	Logs module (last N lines of Conduit log, redaction)
	P3.7	Health check endpoint GET /api/health
	P3.8	Security middleware (CSRF, rate limiting, security headers)

L	Code	Deliverable / Component
	P3.9	Input validation schemas (Pydantic models)
	P3.10	DDNS status endpoint GET /api/ddns/status (reads ddns.log)
1	**P4 — Frontend**	**Single-page web dashboard (HTML/CSS/Vanilla JS, no build step)**
	P4.1	Login page (login.html)
	P4.2	Dashboard layout and navigation (dashboard.html + base CSS)
	P4.3	Node status and control panel (status widget + start/stop/restart buttons)
	P4.4	System health panel (CPU, RAM, temp, disk)
	P4.5	Traffic counter widget
	P4.6	Log viewer panel (last 200 lines, manual refresh)
	P4.7	Settings page (change password, session timeout)
	P4.8	JavaScript polling module (status, metrics every 5–10 s)
	P4.9	DDNS status panel (current IP, last update time, last result)
1	**P5 — Configuration**	**Runtime configuration files and secret templates**
	P5.1	.env.example — secret placeholders including CF_API_TOKEN, CF_ZONE_NAME, CF_RECORD_NAME, SESSION_SECRET
	P5.2	config.example.json — application configuration template
	P5.3	SQLite schema for sessions and metrics
1	**P6 — Test Suite**	**Automated tests providing ≥60% backend coverage**
	P6.1	Unit tests for backend modules (pytest)
	P6.2	Integration tests for API endpoints (pytest + httpx)
	P6.3	Install script syntax check (bash -n)
1	**P7 — Documentation**	**Written materials needed for a public open-source release**
	P7.1	README.md — project description, install instructions, screenshot placeholder
	P7.2	CONTRIBUTING.md — dev setup, coding standards, PR process
	P7.3	CHANGELOG.md — v0.1.0 entry
	P7.4	SECURITY.md — vulnerability disclosure policy
	P7.5	CODE_OF_CONDUCT.md — Contributor Covenant v2.1

3. Work Breakdown Structure (WBS)

The WBS lists every task needed to produce the PBS deliverables. Tasks are grouped by epic (E1–E9). Each task has an effort estimate in developer-days and a list of prerequisite tasks.

ℹ D = developer-days. 1 D = approximately 6–7 focused hours. Ranges (e.g. 0.5–1) reflect uncertainty.

ID	Task	Effort	Depends on
E1	**Project Setup & Repository**	**3–4 D**	—
1.1	Create GitHub repository; configure branch protection on main and develop; add .gitignore, .gitattributes	**0.5 D**	—
1.2	Create GitHub Actions workflow: lint (ruff), test (pytest), security audit (pip-audit), install script syntax check	**1 D**	1.1
1.3	Add GitHub issue templates (Bug Report, Feature Request, Security Vulnerability) and PR template	**0.5 D**	1.1
1.4	Define repository structure; create all top-level directories and placeholder files	**0.5 D**	1.1
1.5	Write developer environment setup guide (local Python venv, how to run tests, how to run the dev server)	**0.5–1 D**	1.4
E2	**Infrastructure & Deployment**	**6–8 D**	—
2.1	Write conduit-cc.service systemd unit (run as conduit-cc user, sandboxing, restart policy)	**0.5 D**	1.4
2.2	Write conduit-cc.conf Nginx virtual host (proxy_pass to 127.0.0.1:8000, security headers, login rate limit). Configure ngx_http_realip_module: real_ip_header CF-Connecting-IP; set_real_ip_from for all Cloudflare IP ranges. HTTP→HTTPS only needed for direct-IP path; Cloudflare handles browser TLS.	**1.5 D**	1.4
2.3	Write docs/pre-install.md (user-facing pre-install checklist: Cloudflare dashboard steps, Origin Cert creation, API token scoping) and docs/tls-setup.md (Cloudflare Origin Cert + nginx config; Let's Encrypt alternative path for direct-IP)	**1.5 D**	2.2
2.4	Write install.sh: [interactive] prompt for CF_API_TOKEN → validate via Cloudflare API (zone lookup); prompt for CF_ZONE_NAME + CF_RECORD_NAME → validate DNS A record exists + proxied:true; prompt for Origin Cert + key paths → validate with openssl; prompt for admin password; write /etc/conduit-cc/.env (640, conduit-cc); configure Nginx server_name from CF_RECORD_NAME; idempotent; post-install summary with dashboard URL	**3–4 D**	2.1, 2.2, 2.3
2.5	Write uninstall.sh (reverse of install.sh; confirm before deleting config)	**0.5 D**	2.4

ID	Task	Effort	Depends on
2.6	Write update.sh (stop service, backup config, pull latest, reinstall deps, run migrations, restart)	**1 D**	2.4
2.7	Integrate cloudflare-ddns.sh using Script B behaviour (preserve proxy status from API, never force override): read CF_* vars from .env; structured JSON log per run; install.sh runs script once immediately after cron setup to verify end-to-end API → DNS update works before completing; add CF_API_TOKEN / CF_ZONE_NAME / CF_RECORD_NAME to .env.example with comments; logrotate config for ddns.log	**0.5 D**	2.4
E3	**Backend — Authentication**	**4–5 D**	—
3.1	Create FastAPI application skeleton: app factory, config loader, .env support (python-dotenv), CORS, error handlers	**1 D**	1.4
3.2	Implement session store: SQLite table; create/validate/expire/destroy session; cryptographically random session ID (32 bytes)	**1 D**	3.1
3.3	Implement auth endpoints: POST /api/auth/login (bcrypt verify, set session cookie), POST /api/auth/logout (destroy session)	**1 D**	3.2
3.4	Implement account lockout: track failed attempts per username; lock for 15 min after 5 failures; ccc-unlock CLI command	**0.5–1 D**	3.3
3.5	Implement session middleware: validate session cookie on every protected route; redirect unauthenticated requests to /login	**0.5 D**	3.2
E4	**Backend — Conduit Adapter**	**4–5 D**	—
4.1	Implement Conduit adapter: call systemd (start/stop/restart/status) via subprocess + systemctl; parse output into structured status	**1.5 D**	3.1
4.2	Implement GET /api/status endpoint: return node status + last-changed timestamp + current systemd state	**0.5 D**	4.1
4.3	Implement POST /api/conduit/{start|stop|restart} endpoints: validate action; call adapter; return new status	**0.5 D**	4.1
4.4	Implement pairing workflow endpoint: accept pairing link transiently; pass to Conduit CLI; never log or store the link; return result	**1.5–2 D**	4.1
E5	**Backend — Metrics & Logs**	**3–4 D**	—
5.1	Implement GET /api/metrics/system: read CPU %, RAM MB/%, CPU temp (°C), disk usage GB/%; use psutil; cache for 5 s	**1 D**	3.1
5.2	Implement GET /api/metrics/traffic: read bytes transferred from Conduit log or metrics file; return upload + download totals	**1 D**	3.1

ID	Task	Effort	Depends on
5.3	Implement GET /api/logs: read last N lines of Conduit service log (journalctl or log file); redact pairing link patterns; return JSON array	**1–2 D**	3.1
5.4	Implement GET /api/ddns/status: parse last 50 lines of /var/log/conduit-cc/ddns.log; extract last update timestamp, public IP, and success/failure; return JSON	**0.5–1 D**	3.1
E6	**Frontend**	**7–9 D**	—
6.1	Create base layout: base.css (typography, colour variables, responsive grid), base.js (fetch wrapper, error toast)	**1 D**	3.1
6.2	Build login page (login.html + login.js): form, validation, error display, redirect on success	**0.5–1 D**	6.1
6.3	Build dashboard shell (dashboard.html): navigation sidebar, page header, content area, logout button	**1 D**	6.1
6.4	Build node status & control panel: status badge (colour-coded), Start/Stop/Restart buttons with confirmation dialog and spinner	**1–1.5 D**	6.3, 4.2, 4.3
6.5	Build system health panel: CPU, RAM, temp, disk gauges; polling every 10 s via setInterval; colour-coded thresholds	**1–1.5 D**	6.3, 5.1
6.6	Build traffic counter widget: upload + download totals; polling every 30 s	**0.5 D**	6.3, 5.2
6.7	Build log viewer panel: display last 200 lines; manual refresh button; auto-refresh every 30 s	**1 D**	6.3, 5.3
6.8	Build settings page: change password form (current + new + confirm); session timeout selector; save with success/error feedback	**1–1.5 D**	6.3
6.9	Build DDNS status panel: show current public IP, hostname (CF_RECORD_NAME), last update time, last result badge (green/red); polls GET /api/ddns/status every 60 s	**0.5 D**	6.3, 5.4
E7	**Security Hardening**	**3–4 D**	—
7.1	Add Nginx security headers to conduit-cc.conf: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, HSTS	**0.5 D**	2.2
7.2	Add CSRF protection to all state-changing API endpoints (double-submit cookie pattern)	**1 D**	3.5
7.3	Add Nginx rate limiting for POST /api/auth/login: 10 req/s per IP; return 429 with Retry-After header	**0.5 D**	2.2
7.4	Audit all API inputs for Pydantic model coverage; ensure no raw request data reaches adapter or filesystem	**1–2 D**	4.1, 5.1, 5.2, 5.3
E8	**Testing**	**4–5 D**	—

ID	Task	Effort	Depends on
8.1	Write unit tests for backend modules: auth (login, lockout, session expiry), adapter (status parsing), metrics (psutil mock), logs (redaction)	**2 D**	3.3, 3.4, 4.1, 5.1, 5.3
8.2	Write integration tests for all v0.1 API endpoints using pytest + httpx test client	**1.5 D**	8.1
8.3	Run install.sh on a fresh Ubuntu 22.04 ARM64 image (Raspberry Pi 4 or VM); document and fix any failures	**0.5–1 D**	2.4
E9	**Documentation**	**3–4 D**	—
9.1	Write README.md: project description, feature list, install instructions, screenshot placeholder, licence and CI badges, link to CONTRIBUTING	**1–1.5 D**	2.4
9.2	Write CONTRIBUTING.md: dev setup, how to run tests, commit message format (Conventional Commits), PR checklist	**0.5–1 D**	1.4
9.3	Write SECURITY.md (disclosure email, response SLA) and CODE_OF_CONDUCT.md (Contributor Covenant v2.1)	**0.5 D**	1.4
9.4	Write CHANGELOG.md v0.1.0 entry: list every implemented feature and known limitation	**0.5 D**	8.3

4. Critical Path

The critical path is the longest chain of dependent tasks. Delays on this path delay the entire v0.1 release.

ℹ Critical path (single developer): 1.1 → 1.4 → 3.1 → 3.2 → 3.3 → 4.1 → 4.4 → 6.1 → 6.3 → 6.4 → 8.1 → 8.2 → 8.3 → 9.1 → 9.4

Phase	Critical Path Task	Effort
Phase 1	1.1 Create repository and branch protection	0.5 D
Phase 1	1.4 Define repository structure	0.5 D
Phase 2	3.1 FastAPI application skeleton	1 D
Phase 2	3.2 Session store (SQLite)	1 D
Phase 2	3.3 Auth endpoints (login/logout)	1 D
Phase 2	4.1 Conduit adapter (systemctl wrapper)	1.5 D
Phase 2	4.4 Pairing workflow endpoint	1.5–2 D
Phase 3	6.1 Base layout CSS + JS	1 D
Phase 3	6.3 Dashboard shell	1 D
Phase 3	6.4 Node status & control panel	1–1.5 D
Phase 4	8.1 Unit tests	2 D
Phase 4	8.2 Integration tests	1.5 D
Phase 4	8.3 Test install.sh on real hardware	0.5–1 D
Phase 4	9.1 README.md	1–1.5 D
Phase 4	9.4 CHANGELOG.md v0.1.0	0.5 D

4.1 Parallelisation Opportunities

If two or more developers work simultaneously, the following task groups can be parallelised after Phase 1 is complete:

After	Developer A	Developer B
Phase 1 done	E3 (Auth) + E4 (Adapter)	E2 (Infrastructure)
E3 done	E6 (Frontend)	E5 (Metrics & Logs) + E7 (Security)
E4–E6 done	E8 (Testing)	E9 (Documentation)

✓ With two developers and parallelisation, v0.1 is achievable in 4–5 weeks elapsed time.

5. GitHub Issue Mapping

Each WBS task maps to one GitHub Issue. The table below shows the mapping, suggested labels, and milestone.

WBS	Issue	Title	Labels	Effort
1.1	#1	Initialize repository structure and branch protection	setup	0.5 D
1.2	#2	Create GitHub Actions CI pipeline	setup, ci	1 D
1.3	#3	Add issue templates and PR template	setup, docs	0.5 D
1.4	#4	Define repository directory structure	setup	0.5 D
1.5	#5	Write developer environment setup guide	docs	0.5 D
2.1	#6	Create systemd service unit (conduit-cc.service)	infrastructure	0.5 D
2.2	#7	Create Nginx virtual host configuration	infrastructure	1 D
2.3	#8	Write TLS setup guides (Cloudflare + Let's Encrypt)	infrastructure, docs	1 D
2.4	#9	Write install.sh installation script	infrastructure	2–3 D
2.5	#10	Write uninstall.sh removal script	infrastructure	0.5 D
2.6	#11	Write update.sh upgrade script	infrastructure	1 D
3.1	#12	Create FastAPI application skeleton	backend	1 D
3.2	#13	Implement SQLite session store	backend, auth	1 D
3.3	#14	Implement login and logout endpoints	backend, auth	1 D
3.4	#15	Implement account lockout and ccc-unlock CLI	backend, auth, security	1 D
3.5	#16	Implement session validation middleware	backend, auth	0.5 D
4.1	#17	Implement Conduit adapter (systemctl wrapper)	backend, conduit	1.5 D
4.2	#18	Implement GET /api/status endpoint	backend, conduit	0.5 D
4.3	#19	Implement start/stop/restart endpoints	backend, conduit	0.5 D
4.4	#20	Implement pairing workflow (transient, no storage)	backend, conduit, security	1.5 D
5.1	#21	Implement GET /api/metrics/system	backend, metrics	1 D
5.2	#22	Implement GET /api/metrics/traffic	backend, metrics	1 D
5.3	#23	Implement GET /api/logs with redaction	backend, logs	1 D
6.1	#24	Build base CSS layout and JS fetch wrapper	frontend	1 D
6.2	#25	Build login page	frontend, auth	0.5 D
6.3	#26	Build dashboard shell and navigation	frontend	1 D
6.4	#27	Build node status and control panel	frontend, conduit	1 D
6.5	#28	Build system health panel	frontend, metrics	1 D

WBS	Issue	Title	Labels	Effort
6.6	#29	Build traffic counter widget	frontend, metrics	0.5 D
6.7	#30	Build log viewer panel	frontend, logs	1 D
6.8	#31	Build settings page (change password)	frontend, auth	1 D
7.1	#32	Add security headers to Nginx config	security, infrastructure	0.5 D
7.2	#33	Implement CSRF protection	security, backend	1 D
7.3	#34	Add login rate limiting in Nginx	security, infrastructure	0.5 D
7.4	#35	Audit Pydantic model coverage for all inputs	security, backend	1 D
8.1	#36	Write backend unit tests (≥60% coverage)	testing	2 D
8.2	#37	Write API integration tests	testing	1.5 D
8.3	#38	Test install.sh on Raspberry Pi 4 hardware	testing, infrastructure	1 D
9.1	#39	Write README.md	docs	1 D
2.7	#41	Integrate Cloudflare DDNS script and cron job	infrastructure, security	0.5 D
5.4	#42	Implement GET /api/ddns/status endpoint	backend, metrics	0.5 D
6.9	#43	Build DDNS status panel on dashboard	frontend, metrics	0.5 D
9.2–9.3	#40	Write CONTRIBUTING.md, SECURITY.md, CODE_OF_CONDUCT.md	docs	1 D

END OF DOCUMENT